ACTIVITY 2: CYBER RANGE

Cyber Range as a Service

Topic: SQL Injection Attack and Web Application Firewall

Submitted By	Poornachandra M
Roll Number	24MSRDF017
Programme	4th Semester MSc DFIS
Activity	Activity 2: Cyber Range
Topic	SQL Injection Attack and Web Firewall
Submitted To	Vishnu Venkatesh Sir

1. Introduction

In today's digitally connected world, web application security has become a critical concern for organizations across all sectors. Among the many threats that target web applications, SQL Injection (SQLi) remains one of the oldest yet most dangerous vulnerabilities, consistently featuring in the OWASP Top 10 list of critical web application security risks.

This activity presents an Attack-Defense Simulation within the conceptual framework of Cyber Range as a Service (CRaaS) — a cloud-hosted, controlled environment designed to allow cybersecurity professionals and students to practice, test, and study real-world attack scenarios without endangering live systems.

The simulation focuses specifically on:

- Modelling SQL Injection attack techniques against a mock vulnerable web application
- Demonstrating how a Web Application Firewall (WAF) detects, classifies, and blocks malicious payloads
- Providing an interactive, educational GUI for hands-on exploration of offensive and defensive strategies

1.1 Motivation

The motivation for selecting SQL Injection as the simulation topic arises from its real-world prevalence and impact. High-profile data breaches — including those affecting government portals, e-commerce platforms, and healthcare databases — have frequently been attributed to unpatched SQL Injection vulnerabilities. Understanding how attackers craft such payloads, and how defences intercept them, is fundamental to any digital forensics or information security curriculum.

2. Conceptual Background

2.1 Cyber Range as a Service (CRaaS)

A Cyber Range is a controlled, isolated digital environment used for the simulation of cyber attacks and defences. Cyber Range as a Service (CRaaS) extends this concept by hosting such environments in the cloud, making them accessible to students, researchers, and security teams on demand, without requiring dedicated physical infrastructure.

Key characteristics of a Cyber Range environment include:

- Isolation: The simulation environment is completely separated from production networks, preventing unintended damage
- Realism: The environment replicates real-world infrastructure — including servers, databases, firewalls, and end-user interfaces
- Interactivity: Participants can actively execute attacks and observe defences in real time
- Logging and Analytics: All events are captured for post-incident analysis and learning

2.2 SQL Injection

SQL Injection is a code injection attack technique that manipulates the SQL queries sent by an application to its underlying database. It occurs when user-supplied input is concatenated directly into a SQL query without proper sanitisation or parameterisation.

A classic example of a vulnerable query:

```
SELECT * FROM users WHERE username = '[input]';
```

If the input is: admin' OR '1'='1' -- the resulting query becomes:

```
SELECT * FROM users WHERE username = 'admin' OR '1'='1' --';
```

Since '1'='1' is always true, this returns all rows from the users table — effectively bypassing authentication entirely.

2.3 Categories of SQL Injection

Category	Mechanism	Example Payload
In-Band SQLi	Error / Union-based; results returned directly in response	`' UNION SELECT * FROM users --`
Blind Boolean	Asks true/false questions; infers data from app behaviour	`' AND 1=1 -- vs ' AND 1=2 --`
Time-Based Blind	Uses DB time-delay functions to infer data	`' AND SLEEP(5) --`
Out-of-Band SQLi	Exfiltrates data via DNS or HTTP side channels	`LOAD_FILE() / UTL_HTTP`
Stacked Queries	Chains multiple SQL statements in one input	`'; DROP TABLE users; --`

2.4 Web Application Firewall (WAF)

A Web Application Firewall is a security layer that monitors, filters, and blocks HTTP/HTTPS traffic to and from a web application. Unlike a traditional network firewall that operates at the IP and TCP/UDP layer, a WAF understands application-layer semantics and can inspect request parameters, headers, cookies, and body content.

WAFs defend against SQLi by maintaining a rule set of known malicious patterns — typically expressed as regular expressions — that match common injection signatures. When a request matches a rule, the WAF can block the request, log the event, or respond with a custom error page.

Modern WAFs also incorporate rate limiting, anomaly scoring (where each suspicious characteristic adds to a cumulative risk score), and machine-learning-based detection to combat novel, obfuscated attacks.

3. Implementation

3.1 Technology Stack

- Language: Python 3.x
- GUI Library: Tkinter (built-in Python standard library)
- Pattern Matching: Python re module (Regular Expressions)
- Hashing: Python hashlib (MD5 for mock password storage demonstration)
- Concurrency: Python threading (non-blocking request processing)

3.2 Architecture Overview

The simulation is structured into four distinct logical layers:

Layer	Description
Presentation	Tkinter GUI — login form, payload panel, log tabs, analytics canvas
WAF Engine	Regex-based rule matching engine (15 active rules). Inspects every request before it reaches the database layer
Query Simulator	Models how a vulnerable PHP/Python web app would construct and execute a raw SQL query — without executing real SQL
Mock Database	In-memory Python dictionary representing users and products tables, used to simulate realistic data leakage scenarios

3.3 WAF Rule Set

The WAF engine implements 15 regex-based detection rules, each targeting a specific attack category:

#	Rule Name	Regex Pattern
01	Classic comment bypass	`'.*--`
02	Boolean-based SQLi	`(\bor\b|\band\b)\s+\d+=\d+`
03	UNION SELECT injection	`union\s+(all\s+)?select`
04	Stacked query injection	`;\s*(drop|delete|insert|update)\b`
05	Time-based blind SQLi	`sleep\s*\(\s*\d+\s*\)`
06	Benchmark time attack	`benchmark\s*\(`
07	Schema enumeration	`information_schema`
08	Hex encoding bypass	`0x[0-9a-f]{2,}`
09	CHAR() encoding bypass	`char\s*\(\s*\d+`
10	Inline comment obfuscation	`/\*.*\*/`
11	File read attempt	`load_file\s*\(`
12	File write attempt	`into\s+(out|dump)file`

13	EXEC injection	`exec\s*\(`
14	Command shell injection	`xp_cmdshell`
15	MSSQL time delay	`waitfor\s+delay`

3.4 Key Python Components

3.4.1 WAF Inspection Function

The core defence function iterates over all compiled regex patterns and returns the first match:

```
def waf_inspect(payload: str):
    for pattern, desc in WAF_COMPILED:
        if pattern.search(payload):
            return True, desc   # blocked, reason
    return False, 'CLEAN'
```

3.4.2 Vulnerable Query Simulator

This function models the behaviour of an unsanitised login endpoint without executing any real database query. It returns the raw query string that would have been constructed, and the simulated result set based on pattern matching:

```
def simulate_vulnerable_query(username_input):
    raw_query = f"SELECT * FROM users WHERE username = '{username_input}'"
    # Checks for tautology, UNION, comment bypass patterns
    # Returns (query_string, matching_rows)
```

3.4.3 GUI Architecture (CyberRangeApp)

The GUI class inherits from tk.Tk and is composed of:

- Left Panel: Simulated login form, sample payload library, and live WAF rule display
- Right Panel: Tabbed notebook with WAF Log, Query Simulation, DB Response, and Analytics tabs
- Status Bar: Real-time counters for total requests, blocked, and allowed
- Analytics Canvas: Bar chart updated after each request using tk.Canvas drawing primitives

4. Working Process

4.1 Request Lifecycle

Every request submitted through the GUI follows this processing pipeline:

1. User enters a username or SQLi payload in the input field
2. The Send Request button triggers the _execute_attack() method
3. A background thread is spawned via threading.Thread to keep the UI responsive
4. The WAF engine (waf_inspect) inspects the payload against all 15 regex rules

5. If WAF is ENABLED and a rule matches: the request is blocked; the WAF Log tab records the payload, timestamp, and matched rule; the Query and DB tabs show the request never reached the database
6. If WAF is DISABLED or no rule matches: the payload is forwarded to the Query Simulator; the simulated raw SQL query is displayed; the DB Response tab shows the resulting data rows
7. Statistics counters and the analytics chart are updated after every request

4.2 Attack Scenarios Demonstrated

Scenario	Payload	Outcome (WAF ON)
Authentication Bypass	`admin' OR '1'='1' --`	Blocked — Tautology rule #2
Data Dump via UNION	`' UNION SELECT * FROM users --`	Blocked — UNION SELECT rule #3
Stacked DROP TABLE	`admin'; DROP TABLE users; --`	Blocked — Stacked query rule #4
Time-Based Blind	`admin' AND SLEEP(5) --`	Blocked — SLEEP() rule #5
Schema Enumeration	`' UNION SELECT table_name FROM information_schema.tables --`	Blocked — Schema rule #7
Hex Encoding Bypass	`0x61646d696e`	Blocked — Hex encoding rule #8
Legitimate Login	`alice`	Allowed — No rules matched

4.3 WAF Toggle — Demonstrating Impact

A key educational feature of the simulation is the WAF toggle switch. When the WAF is disabled, the same malicious payloads that were previously blocked are forwarded to the Query Simulator, which demonstrates their destructive potential:

- Authentication bypass — all user records are returned, simulating unauthorised access
- UNION injection — a fabricated LEAKED_DATA row is returned, demonstrating data exfiltration
- Stacked queries — the raw DROP TABLE statement is shown in the query log, illustrating irreversible data destruction

This comparison makes the value of the WAF immediately tangible to students.

5. Overall Functionality

5.1 GUI Panels Summary

- Login Form: Accepts freeform text input; supports manual typing or one-click loading of sample payloads
- Sample Payload Library: 10 pre-loaded payloads covering the major SQLi categories; colour-coded red for attacks, green for legitimate inputs
- WAF Rule Display: Scrollable read-only view of all 15 active WAF rules
- WAF Log Tab: Timestamped log of every request, WAF decision, and matched rule name

- Query Simulation Tab: Displays the raw SQL query that would have been constructed — identical to what a real vulnerable backend would produce
- DB Response Tab: Shows the simulated rows returned from the mock database, demonstrating data leakage when attacks succeed
- Analytics Tab: Live bar chart showing the proportion of blocked vs allowed requests; scrollable history table of the last 20 requests

5.2 Educational Value

The simulation achieves the following educational objectives:

8. Offensive Awareness: Students observe exactly how SQL injection payloads are structured and why they work against vulnerable queries
9. Defensive Understanding: The WAF rule set teaches students the principle of signature-based detection and its role in application security
10. WAF Limitations: By exploring obfuscation techniques (hex encoding, CHAR() bypass, inline comments), students understand that WAFs are not infallible and must be complemented by secure coding practices
11. Incident Response: The real-time logging mimics the kind of evidence a security analyst would examine in a post-incident investigation

5.3 Limitations and Future Enhancements

Limitations of the current simulation:

- The WAF uses static regex rules and does not implement anomaly scoring or machine-learning detection
- The mock database is entirely in-memory; no actual database engine is involved
- Out-of-band SQLi (DNS exfiltration) is described conceptually but not simulated

Potential future enhancements:

- Integration with an actual SQLite database to simulate real query execution in a sandboxed context
- Addition of anomaly-based scoring alongside signature matching
- Second-order SQLi simulation (stored payloads executed upon retrieval)
- Export of logs to JSON/CSV for post-session analysis
- Multi-user scenario simulation for collaborative attack-defence exercises

6. Conclusion

This activity successfully demonstrates the fundamental concepts of SQL Injection attack and Web Application Firewall defence within a Cyber Range as a Service framework. The GUI-based simulation provides an interactive, visual, and educational environment in which students can experiment with real-world attack payloads and immediately observe their interception by the WAF engine.

The simulation illustrates that while WAFs provide an important and effective layer of defence, they must be complemented by secure application development practices — particularly parameterised queries and prepared statements — which eliminate SQL injection vulnerabilities at their root cause.

The Cyber Range paradigm offers a risk-free space to develop offensive and defensive cybersecurity skills. This activity demonstrates how even a lightweight, software-based simulation can convey the mechanics of one of the most persistent and damaging web application vulnerabilities known to the security community.

7. References

- OWASP Foundation. (2021). OWASP Top Ten — A03:2021 Injection. https://owasp.org/Top10/
- Clarke, J. (2012). SQL Injection Attacks and Defense. Syngress / Elsevier.
- NIST. (2020). National Vulnerability Database — CWE-89: Improper Neutralization of Special Elements in SQL Commands. https://nvd.nist.gov/
- ModSecurity. (2024). OWASP Core Rule Set (CRS) Documentation. https://coreruleset.org/
- Python Software Foundation. (2024). tkinter — Python Interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html
- Portswigger Web Security Academy. (2024). SQL Injection. https://portswigger.net/web-security/sql-injection